Recitaion 06

Function

Exercise 1 | Predict the output

Program 1	Output 1

`def f():`
`    global s`
`    print(s)`
`    s = "more"`
`    return s`
`    s = "less"`
`s = "less"`
`print(f())`
`print(s)`	

Program 2	**Output 2**

`def calc(op, x, y):`
`    return op(x, y)`
`def add(a, b):`
`    return a + b`
`def div(a, b):`
`    if b != 0:`
`        return a/b`
`    print('Denominator was 0.')`
 	
`print(cal(add, 2, 3))`
`print(cal(div, 2, 0))`	

Program 3	**Output 3**

`def foo(x, y):`
`    a += 1`
`    x = a`
`    y = a**2`
`    return x, y`
`a = 2`
`x, y = foo('a', 'b')`
`print(a, b, x, y)`	

Exercise 2 | Property Tax

A city collects property taxes on the assessment value of property, which is 60 percent of the property's actual value. For example, if an acre of land is valued at $10,000, its assessment value is $6,000. The property tax is then 72¢ for each $100 of the assessment value. The tax for the acre assessed at $6,000 will be $43.20.

Write a program that asks for the actual value of a piece of property and calls a function `computePropertyTax` to calculate and display the assessment value and property tax.

Exercise 3 | Painting cost

A painting company has determined that for every 112 square feet of wall space, one gallon of paint and eight hours of labor will be required. The company charges $35.00 per hour for labor.

Write a program that asks the user to enter the square feet of wall space to be painted and the price of the paint per gallon. The program should call the following functions to calculate and display results:

- `paint_volume` computes the number of gallons of paint required
- `labor_hours` computes the hours of labor required
- `paint_cost` computes the cost of the paint
- `labor_cost` computes the labor charges
- `total_cost` computes the total cost of the paint job

Exercise 4 | Maximum of Two Values

Write a function named `max_of_two_numbers` that accepts two integer values as arguments and returns the value that is the greater of the two.

For example, if 7 and 12 are passed as arguments to the function, the function should return 12 . Use the function in a program that prompts the user to enter two integer values. The program should display the value that is the greater of the two.

Exercise 5 | Test Average and Grade

Write a program that asks the user to enter five test scores. The program should display a letter grade for each score and the average test score. Write the following functions in the program:

- `calc_average` - This function should accept five test scores as arguments and return the average of the scores.
- `determine_grade` - This function should accept a test score as an argument and return a letter grade for the score based on the following grading scale:

Exercise 6 | Combination

Write a function called `combination(n, r)` that returns the number of ways to choose r items from n different items, where order does not matter.
Use the formula:

$$C(n,r) = \frac{n!}{r!\,(n-r)!}$$

Here, $n! = 1 \times 2 \times \cdots \times n$, and $0! = 1$).
Requirements:

1. Write a function called `factorial(n)` that returns n!.
2. Call `factorial()` inside `combination(n, r)` to calculate the result.
3. Assume that n and r are integers and `0 ≤ r ≤ n.`
4. Return an integer. Do not use the math module.

Example:

```
print(combination(5, 2))  # 10
print(combination(6, 3))  # 20
print(combination(4, 0))  # 1
```

Exercise 7 | Sum of Digits

Write a program that asks the user to enter a series of single-digit numbers with nothing separating them. The program should display the sum of all the single digit numbers in the string.

For example, if the user enters 2514 , the program should return 12 , which is the sum of 2, 5, 1, and 4.

Exercise 8 | Date Printer

Write a program that reads a string from the user containing a date in the form `mm/dd/yyyy.` It should print the date in the form `March 12, 2014`.

Exercise 9 | Sentence Capitalizer

Write a program with a function that accepts a string as an argument and returns a copy of the string with the first character of each sentence capitalized. For instance, if the argument is "`hello my name is Joe. what is your name?`" the function should return the string "`Hello my name is Joe. What is your name?`"

The program should let the user enter a string and then pass it to the function. The modified string should be displayed.

Exercise 10 | Most Frequent Character

Write a program that lets the user enter a string and displays the character that appears most frequently in the string. Add to your program a function that returns the number of vowels that the string contains.
Hint: you can use the `count()` *method provided for string objects.*

Exercise 11 | Armstrong Numbers

Write a program to find Armstrong numbers in a range and calculate their count, sum, and mean. Use integers, loops, and functions. A non-negative integer with k digits is an Armstrong number if it equals the sum of its digits raised to the power k. Treat 0 as a one-digit number.
For example, `153 = 1 ** 3 + 5 ** 3 + 3 ** 3`.
Also, `1634 = 1 ** 4 + 6 ** 4 + 3 ** 4 + 4 ** 4.`

Complete each function in the following program framework

```
def count_digits(n):
    # TODO: Count digits using a while loop and // 10.
    pass

def digit_power_sum(n):
    # TODO: Extract digits and add each digit ** k.
    pass

def is_armstrong(n):
    # TODO: Check whether n is an armstrong. Return a boolean.


def report_armstrong(start, end):
    # TODO:
    # 1. Show all armstrong between start and end
    # 2. Calculate and print the mean to 2 decimals.
    pass

def main():
    start = int(input("Enter start: "))
    end = int(input("Enter end: "))
    Call report_armstrong().

main()
```

Exercise 12 | Chinese ID number check

Write a program to check the format and checksum of an 18-character Chinese resident ID number, then extract its date of birth and sex code. Rules and steps:

1 Check the format

Write `has_valid_format(id_number).` Require exactly 18 characters. The first 17 must be digits 0 to 9. The final character must be a digit or X.

2 Calculate the checksum

Write `calculate_check_digit(first17).` Assume `first17` contains exactly 17 digits. For each index `i` from 0 to 16, calculate `weight = (2 ** (17 - i)) % 11`. Multiply `int(first17[i])` by this weight and add it to a running total. Let `remainder = total % 11`. If `remainder` is 10, the checksum is letter 'X', otherwise the checksum is `remainder.`

3 Compare the check digit

Write `has_valid_checksum(id_number).` Call `calculate_check_digit()` with the first 17 characters and compare the result with the final character. If they are equal, then the id number is valid. Otherwise is invalid.

4 Extract information

Write `get_birth_date(id_number)` to return a string in `YYYY-MM-DD` format. Characters at indexes 6 to 13 contain `YYYYMMDD`. Write `get_sex(id_number):` the digit at index 16 is odd for "Male" and even for "Female".

5 Complete the main function

Read input as a string, remove surrounding spaces, and convert x to X. Check the format first and the checksum second. Print an error and stop if either check fails. Otherwise, print "Format and checksum passed.", the birth date, and the sex code.

Example:

```
Input: 00000020000101038X
Format and checksum passed.
Date of birth: 2000-01-01
Sex: Female
```

Program framework

Complete each function and TODO, add lines as needed.

```
def has_valid_format(id_number):
    pass

def calculate_check_digit(first17):
    pass


def has_valid_checksum(id_number):
    pass

def get_birth_date(id_number):
    pass

def get_sex(id_number):
    pass

def main():
    id_number = input("Enter an ID number: ").strip().upper()
    # TODO: Check format; print an error and return if needed.
    # TODO: Check checksum; print an error and return if needed.
    print("Format and checksum passed.")
    print("Date of birth:", get_birth_date(id_number))
    print("Sex:", get_sex(id_number))
main()
```